Infrastructure as Code – Introduction

WELCOME TO CLASS 2!

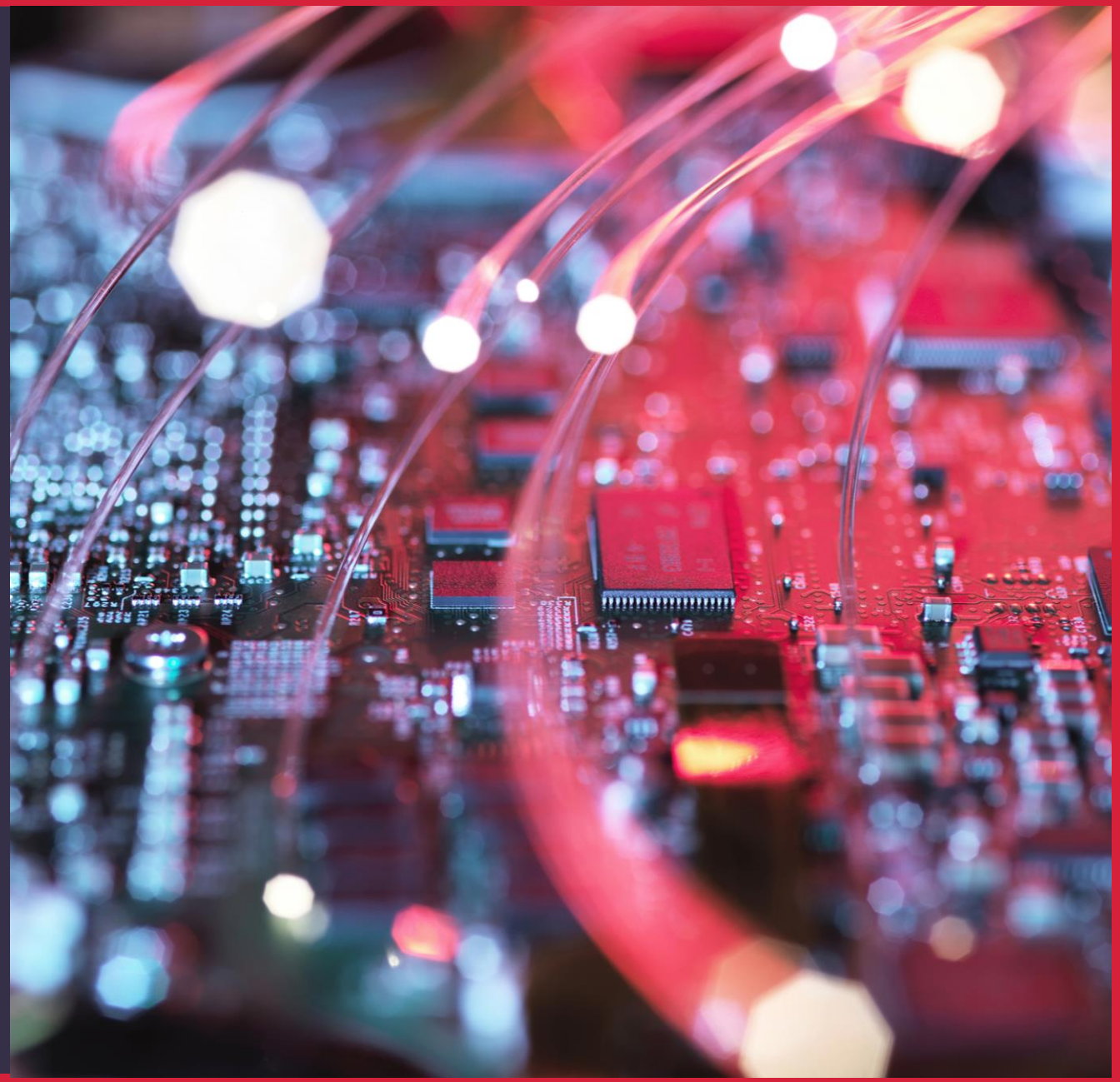


Photo by David Becker on Unsplash

Agenda

- What is Infrastructure as Code?
- IaC options and tool selection for this course.
- **Break**
- Install Terraform and verify that it is working.
- Enable create access for your Terraform instance to your AWS instance.
- Use Terraform to create some very simple AWS infrastructure.
- Verify that the infrastructure is created successfully through the AWS console.

Goals of the day

- What is Infrastructure as Code?
- Install Terraform & AWS CLI
- Create an EC2 using Terraform

https://www.redhat.com/sysadmin/pros-and-cons-infrastructure-code

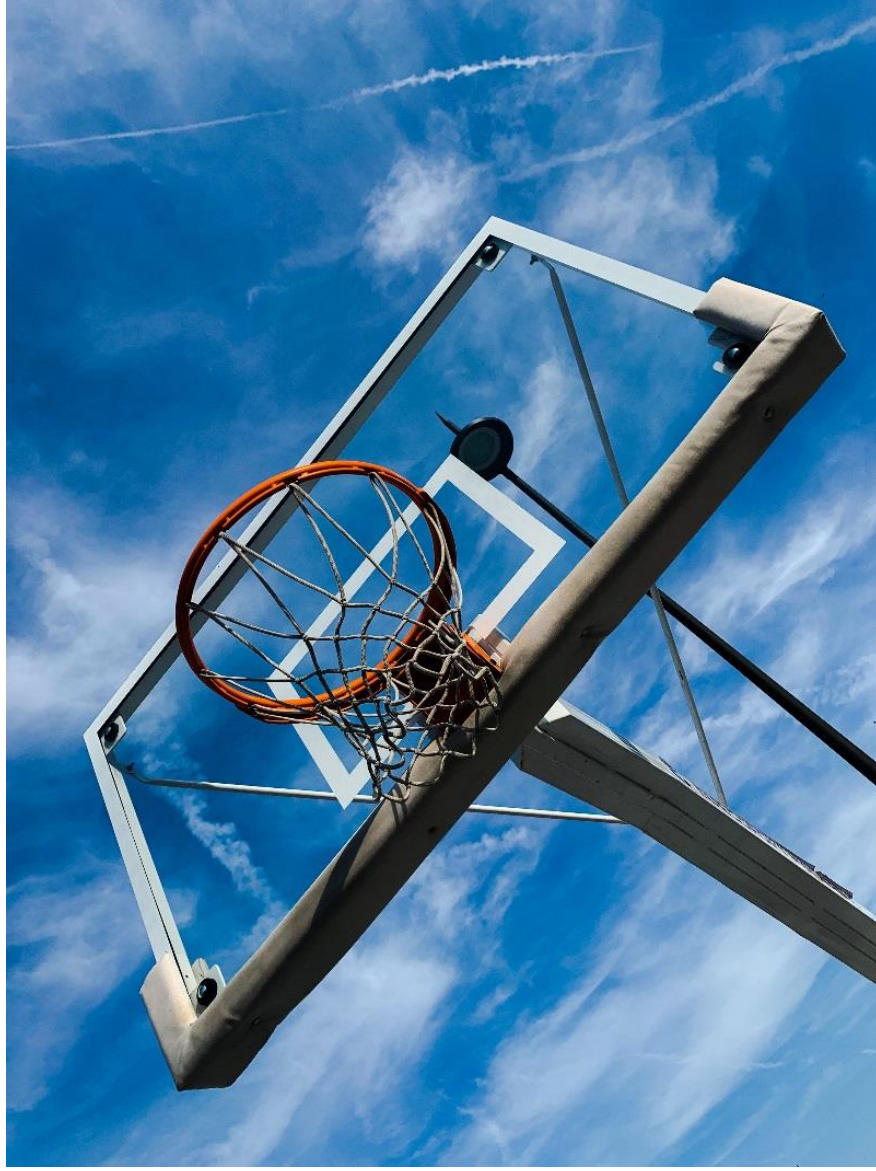
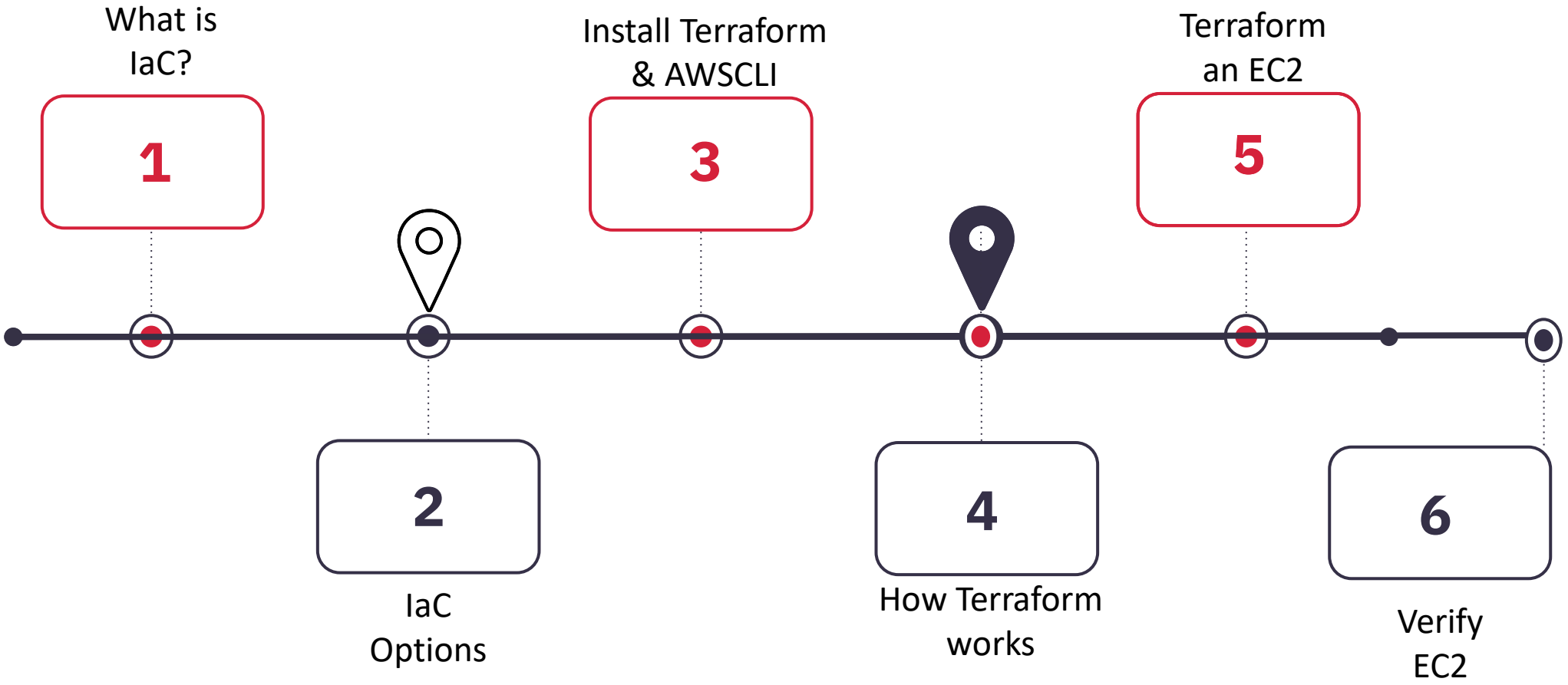


Photo by David Becker on Unsplash

"Infrastructure as code is an approach to managing IT infrastructure for the age of cloud, microservices and continuous delivery."

- Kief Morris, head of continuous delivery for ThoughtWorks Europe

Journey Map


What is IaC?
1
Install Terraform & AWSCLI
3
Terraform an EC2
5
2
IaC Options
4
How Terraform works
6
Verify EC2

What is Infrastructure as Code (IaC)? (1 of 3)

As a CloudOps engineer, IaC is one of the most important things you will learn.

Many of the best practices that have evolved to support the creation of reliable, high-quality software can be applied to IaC, including;

- Source code management – branching, tagging, pull requests
- Quality Assurance
- Modular design to promote reusability

What is Infrastructure as Code (IaC)? (2 of 3)

The following benefits apply to IaC

- Streamlines the previously error prone and time-consuming practice of manually creating infrastructure.
- Enables reliable and repeatable infrastructure creation.
- Facilitates high levels of parity between production and non-production environments
- Leads to more dependable outcomes in software delivery.
- Reduces cost. *

* https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code-iac

What is Infrastructure as Code (IaC)? (3 of 3)

The following benefits apply to IaC

- A well designed IaC implementation enables a relatively small team to manage a very large infrastructure estate very effectively
- Improves turnaround times for security vulnerabilities
- Improves infrastructure currency due to the ease with which new infrastructure can be created and the old removed.
- Enables greater experimentation in the organization due to the options IaC opens up.

Infrastructure as Code Options (1 of 7)

You have probably heard of some of the following IaC tools:

- Terraform
- Ansible
- CloudFormation
- Chef
- Puppet

How do they differ and how do we decide which tools to adopt?

Infrastructure as Code Options (2 of 7)

Here are some of the common categories commonly used in assessing IaC options

- mutable vs immutable
- declarative vs imperative
- provisioning vs configuration management
- open or closed source

We will discuss each of these items over next few slides. Please call out what you see as benefits and drawbacks of each choice.

* https://www.ibm.com/topics/infrastructure-as-code

Infrastructure as Code Options (3 of 7)

Mutable meaning liable to change. Infrastructure that can be updated through ad hoc changes, not just through code. Once this happens the code no longer represents the state of the infrastructure.

Immutable cannot be changed once provisioned. Any changes are made through code and the resources is destroyed and provisioned again based on the new code. The code represents the state of the infrastructure, which is a key benefit of IaC.

* https://www.ibm.com/topics/infrastructure-as-code

Infrastructure as Code Options (4 of 7)

Declarative where the desired state of the infrastructure is defined by the IaC and the tooling ensures that this state is achieved. This approach can be considered idempotent – no matter how many times you execute the code, the same state will be achieved. *

Imperative is where exact instructions for how to achieve the desired state is provided and these instructions are executed by the tool. If the script is re-run or fails, there is no guarantee of idempotency.

* https://www.bmc.com/blogs/idempotence/

Infrastructure as Code Options (5 of 7)

Configuration management tools can usually do some degree of provisioning and vice versa, so there is some overlap.

Provisioning: creates and sets up infrastructure through automation, enabling greater scale, speed and consistency in the infrastructure.*

Configuration Management: used to document, maintain and apply change management to configuration updates in infrastructure, using automation and at scale.**

* https://www.redhat.com/en/topics/automation/what-is-provisioning
** https://www.redhat.com/en/topics/automation/what-is-configuration-management

Infrastructure as Code Options (6 of 7)

Code sourcing model also helps give insight to tooling; the applicability across providers, the opportunity to become involved in enhancing tooling, insight into community support.

Closed: also termed as proprietary software e.g. CloudFormation, developed by and used with AWS only.

Open: created by a community of developers and contributors. Insight into product available based on number of contributors, frequency of releases, stars on GitHub.

Break (15 Minutes)

Infrastructure as Code Options (7 of 7)

	Terraform	Ansible	Cloud Formation	Chef	Puppet
Type	Provisioning	Config Mgmt	Provisioning	Config Mgmt	Config Mgmt
Infra	Immutable	Mutable	Immutable	Mutable	Mutable
Paradigm	Declarative	Imperative	Declarative	Imperative	Imperative
Source	Open	Open	Closed	Open	Open
Community	Huge	Huge	AWS	Large	Large
Cloud	All	All	Small	All	All

Adapted from Terraform Up & Running, 3rd Edition, Brikman, Yevgeniy, O'Reilly, 2022

Terraform and Ansible are the clear leaders in Provisioning and Configuration Management respectively, based on Community support alone.
We will focus on Terraform first and work through provisioning of infrastructure of increasing complexity in AWS to demonstrate the capability of this tool and how it can be most effective when the code is well designed and managed.
We will also work through several Ansible playbooks to understand how it works.

Terraform Install (1 of 2)

Installation of Terraform is the most important thing you will do in this module.

This information if also available on Moodle

Installation on Mac OS

- /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install.sh)"
- export PATH=/opt/homebrew/bin:$PATH
- brew tap hashicorp/tap
- brew install hashicorp/tap/terraform
- brew update
- brew upgrade hashicorp/tap/terraform
- terraform –version

```
shane@Shanes-MacBook-Pro terraform % brew install hashicorp/tap/terraform
==> Fetching hashicorp/tap/terraform
==> Downloading https://releases.hashicorp.com/terraform/1.5.2/terraform_1.5.2_d
######################################################################## 100.0%
==> Installing terraform from hashicorp/tap
🍺  /opt/homebrew/Cellar/terraform/1.5.2: 3 files, 67.9MB, built in 2 seconds
==> Running `brew cleanup terraform`...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP.
Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).
Removing: /Users/shane/Library/Caches/Homebrew/terraform--1.4.4... (19.3MB)
==> `brew cleanup` has not been run in the last 30 days, running now...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP.
Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).
Removing: /Users/shane/Library/Caches/Homebrew/docutils--0.19... (486.3KB)
Removing: /Users/shane/Library/Caches/Homebrew/pcre2--10.42... (2.0MB)
Removing: /Users/shane/Library/Caches/Homebrew/xz_bottle_manifest--5.4.0... (7.2KB)
Removing: /Users/shane/Library/Caches/Homebrew/sqlite_bottle_manifest--3.40.1... (7.9KB)
Removing: /Users/shane/Library/Caches/Homebrew/awscli_bottle_manifest--2.9.11... (17.3KB)
Removing: /Users/shane/Library/Caches/Homebrew/azure-cli_bottle_manifest--2.43.0... (21.1KB)
Removing: /Users/shane/Library/Caches/Homebrew/pcre2_bottle_manifest--10.42... (8.5KB)
Removing: /Users/shane/Library/Caches/Homebrew/python@3.10_bottle_manifest--3.10.9... (21.6KB)
Removing: /Users/shane/Library/Caches/Homebrew/git_bottle_manifest--2.39.0... (12.8KB)
Removing: /Users/shane/Library/Caches/Homebrew/python@3.11_bottle_manifest--3.11.1... (21.7KB)
Removing: /Users/shane/Library/Caches/Homebrew/docutils_bottle_manifest--0.19-2... (2.6KB)
Removing: /Users/shane/Library/Caches/Homebrew/terraform_bottle_manifest--1.3.9... (7KB)
Removing: /Users/shane/Library/Caches/Homebrew/Cask/hashicorp-vagrant--2.3.4.dmg... (58.5MB)
Removing: /opt/homebrew/lib/python3.11/site-packages/__pycache__/py.cpython-311.pyc... (416B)
Removing: /opt/homebrew/lib/python3.10/site-packages/__pycache__/py.cpython-310.pyc... (303B)
Pruned 0 symbolic links and 2 directories from /opt/homebrew
shane@Shanes-MacBook-Pro terraform % brew update
Already up-to-date.
shane@Shanes-MacBook-Pro terraform % terraform --version
Terraform v1.5.2
on darwin_arm64
```

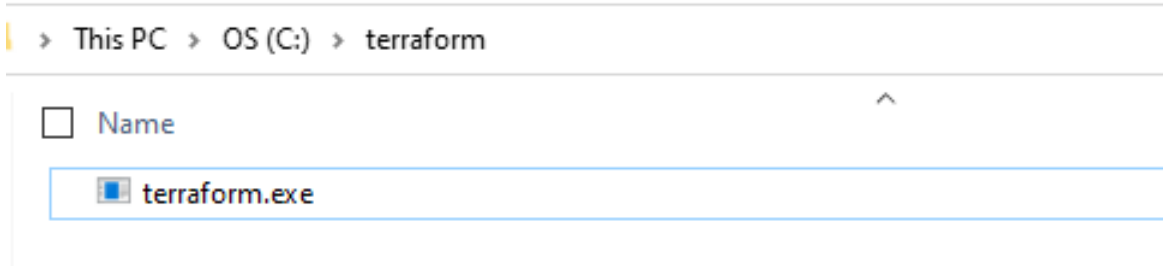
Terraform Install (2 of 2)

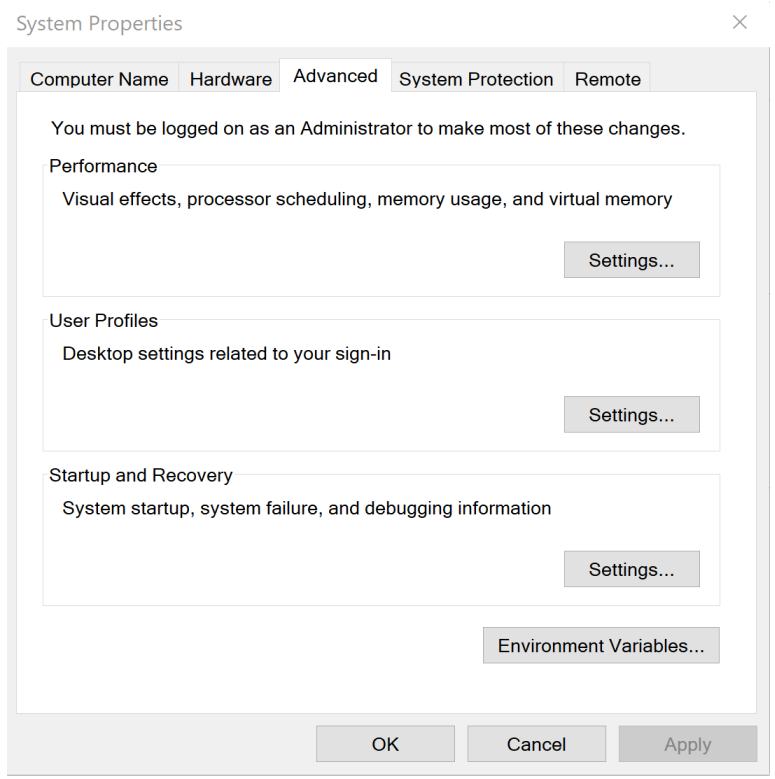
Installation on Windows OS

- Download terraform from https://developer.hashicorp.com/terraform/downloads
- Extract the zip file content to c:\terraform
- Go to System Properties -> Environment Variables
- In System Variables edit the Path
- Select New and add c:\terraform
- Now open Command Prompt (CMD) and type

```
terraform --version
```

- You should have the latest Terraform version 1.5.2 or later





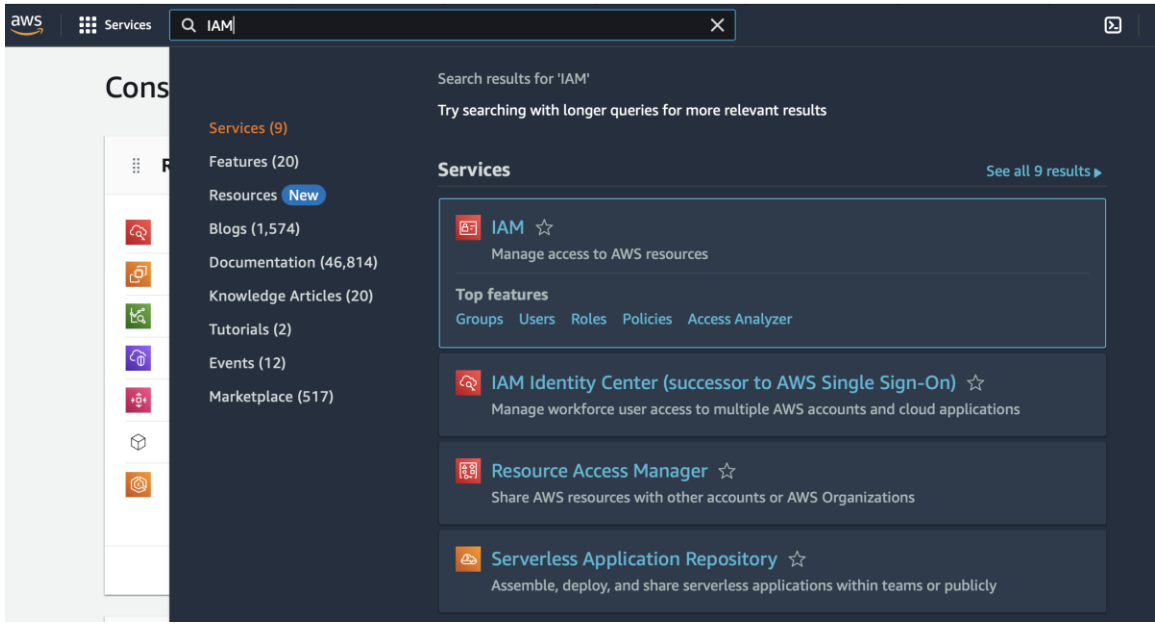
AWS CLI install (1 of 3)

In addition to Terraform install, you also need AWS CLI, which is AWS' command line interface.

If you have not already done so, then create an AWS Free Tier account (instructions in Moodle).

Next search for IAM in search bar (top right of screen). Click through to IAM, then click on 'Manage Access Keys'.

Scroll down to Access keys and 'Create access key'. Ignore warnings on create keys for root users. Best practice is to define a role for AWS CLI, but beyond scope of this module.



AWS CLI install (2 of 3)

Copy Access and Secret access keys and store securely.

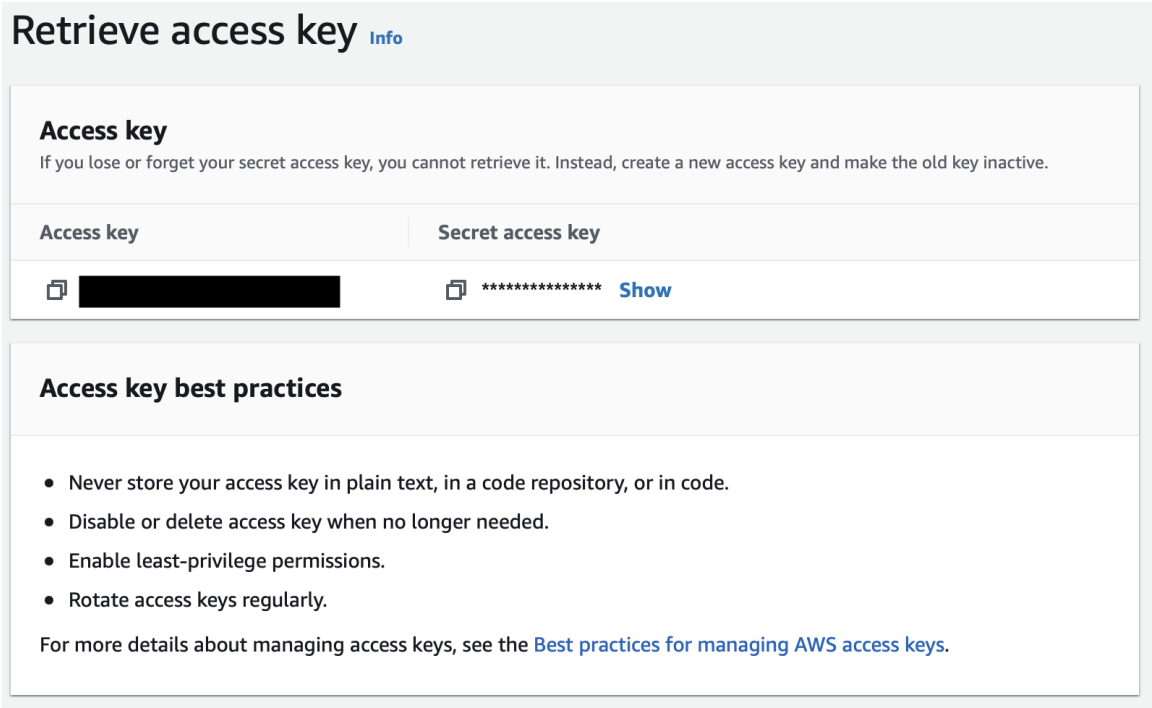
Install AWS CLI, using instructions here..

https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

Open Command Prompt/Terminal: `aws configure`

Copy and paste the keys you saved previously:

- Access Key ID: `blablablaxsekfnbhwfbkfbuers`
- Secret Access Key: `blablablau3brfhvrjelngfjlvwe`
- Default region name: `OK to leave empty <Enter>`
- Default output format: `OK to leave empty <Enter>`



AWS CLI install (3 of 3)

After installation completion open Command Prompt/Terminal and type aws --version

You should have the latest AWS CLI version, greater than 2.9.x.

AWS CLI can be used to enquire on, create and delete resources in your account.

If you are from a Linux Admin background, for example, using the CLI may be faster and more efficient than a GUI.

Terraform will use this install to connect to AWS.

```
shane@Shanes-MacBook-Pro terraform % aws --version
aws-cli/2.9.11 Python/3.11.1 Darwin/21.6.0 source/arm64 prompt/off
shane@Shanes-MacBook-Pro terraform % aws s3 ls
2011-04-12 07:49:35 shanemannion
2020-07-21 11:29:01 shanemannion-training
2019-10-14 08:14:40 shanemannion-video
shane@Shanes-MacBook-Pro terraform % aws ec2 describe-instances
{
    "Reservations": [
        {
            "Groups": [],
            "Instances": [
                {
                    "AmiLaunchIndex": 0,
                    "ImageId": "ami-0d52ddcdf3a885741",
                    "InstanceId": "i-0b5e281399e70eed8",
                    "InstanceType": "t2.medium",
                    "KeyName": "jenkins",
                    "LaunchTime": "2023-05-22T13:26:45+00:00",
                    "Monitoring": {
                        "State": "disabled"
                    },
                    "Placement": {
                        "AvailabilityZone": "us-east-1a",
                        "GroupName": "",
                        "Tenancy": "default"
                    },
                    "PrivateDnsName": "",
                    "ProductCodes": [],
                    "PublicDnsName": "",
                    "State": {
                        "Code": 48,
                        "Name": "terminated"
                    },
```

How does Terraform work? (1 of 3)

At this point, you have installed Terraform on your laptop.

You have also configured a connection from your laptop to you Cloud Platform, AWS in this case. You can also install Azure CLI or gcloud CLI.

The piece in the middle is the Terraform provider, which you specify in your Terraform code.

How does Terraform work? (2 of 3)

Terraform has over 3,000 integrations from over 250 partners as part of their provider ecosystem.

Providers are freely available through the Terraform Registry.

Providers are created by Hashicorp, by Partners (like AWS), by the community or privately by companies.

Providers have also been created for On Prem infra, to allow benefits of IaC to be realized across the full infra domain.

How does Terraform work? (3 of 3)

Provider details are specified in terraform.

The provider is retrieved from the Hashicorp Registry, with the latest version shown.

When we initialize the code, it downloads the provider if required.

To upgrade provider, first delete **.terraform.lock.hcl**

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```
shane@Shanes-MacBook-Pro 01.CreateEC2 % terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.6.1...
- Installed hashicorp/aws v5.6.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

aws

Official by: HashiCorp

Public Cloud

Lifecycle management of AWS resources, including EC2, Lambda, EKS, ECS, provider is maintained internally by the HashiCorp AWS Provider team.

VERSION	PUBLISHED	SOURCE CODE
5.6.1	13 hours ago	hashicorp/terraform-provider-aws

What have we covered so far today?

- ✓ What is IaC?
- ✓ How to evaluate the many IaC options?
- ✓ Why do we focus on Terraform?
- ✓ Install Terraform and AWS CLI
- ✓ How does Terraform work?
- ❑ Overview of most important Terraform commands
- ❑ Example of Terraform in practice

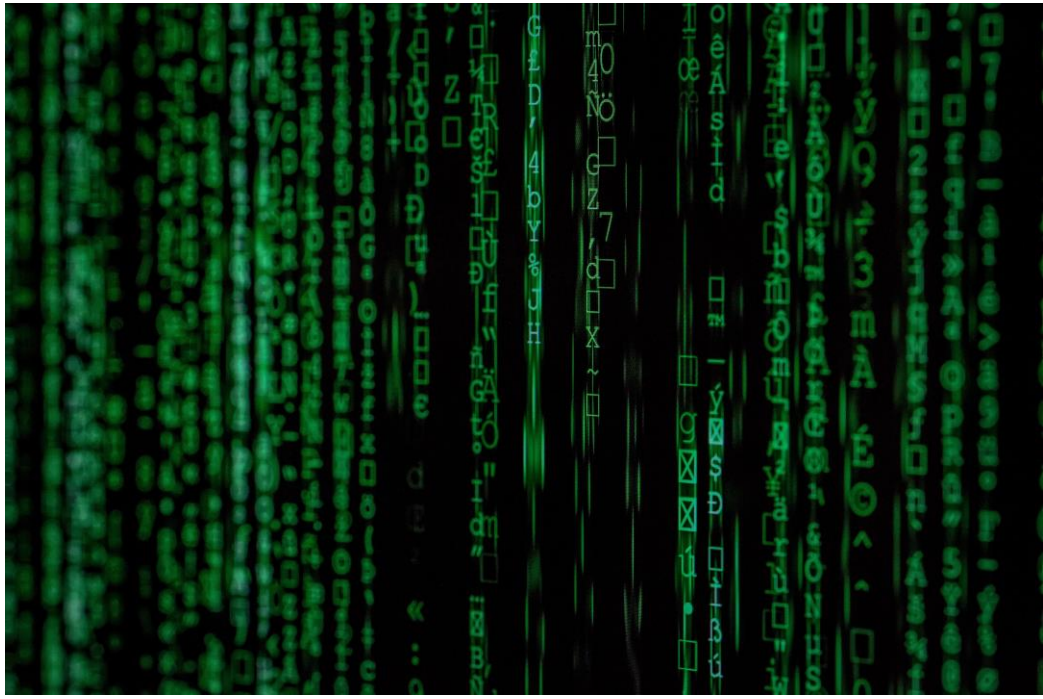


Photo by Markus Spiske on Unsplash

terraform init

This is the first command you will use. It is safe to run many times. In attempting to initialize the working directory, it performs the following functions:

1. Assess backend configuration, defaulting to a local state file, but optionally can reference a Cloud storage location for state management. Detailed review of this topic later in the next class
2. Download and install provider plugins and load any specified provider credentials. Adds provider information (including versions) to dependency lock file, which should be committed to SCM
3. Retrieves source for modules that are included, either in an SCM or in the public Terraform Registry
4. Checks the code for any syntax errors and validates them

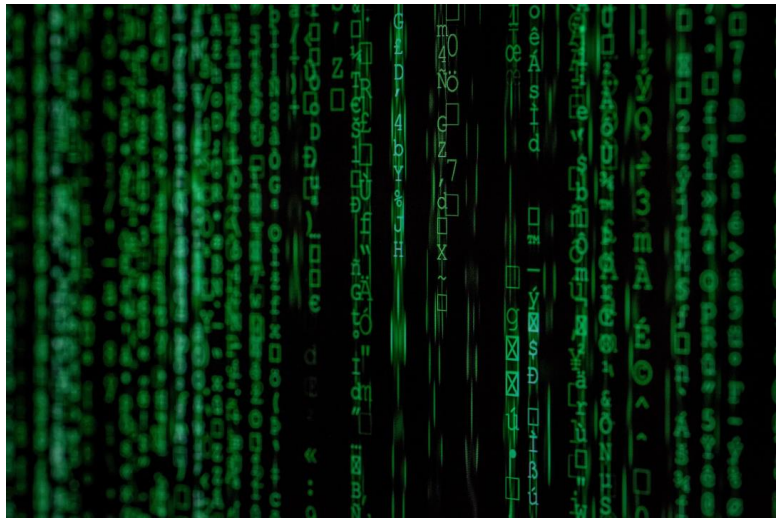


Photo by Markus Spiske on Unsplash

terraform plan

This is the second command you will use. By creating an execution plan, it allows you to preview the changes which this code will make to your infrastructure.

1. When run for the first time, it highlights how many elements will be created by the code
2. When run subsequently, it highlights how any changes to the code will modify the existing elements, or in the case where the infrastructure has changed but the code has not, it will highlight this drift between configuration and reality, making this command suitable for drift prevention
3. Can also run in refresh-only mode to ingest into the state any changes made to remote objects
4. Input variables can be provided to this command as a file or as individual variables
5. A plan file can be created that is then used by apply to create an exact configuration, despite any changes that may have been made to the infrastructure since the plan was created
6. This command is a subset of both `apply` and `destroy`

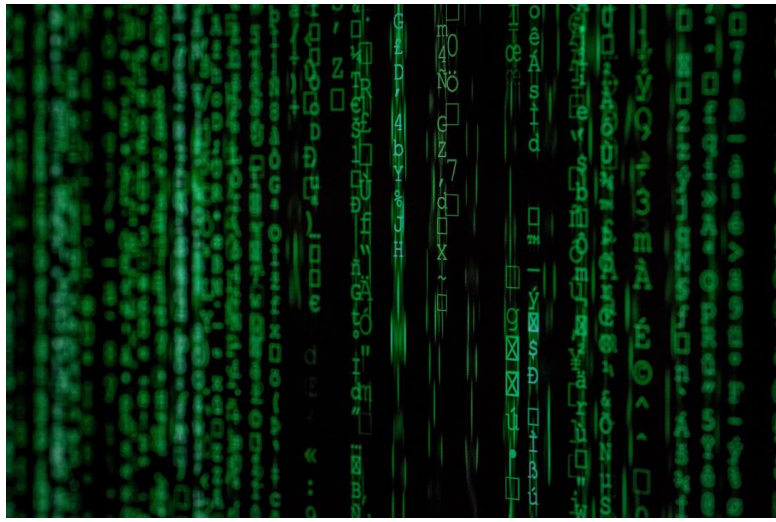


Photo by Markus Spiske on Unsplash

terraform apply

This is the third command you will use. It creates the infrastructure based on the configuration or plan provided.

1. Current state of infrastructure is compared with declared state and any update, deletes or creates needed are then applied to achieve this declared state
2. When a plan is provided, this command will implement without requesting any further approvals
3. Where no plan is provided, one is generated, and the user must approve before the requested infrastructure is provisioned. An auto-approve flag can override but should be used carefully
4. Output from apply can be codified to provide information on the infrastructure that was created, including server and EC2 details or application URLs

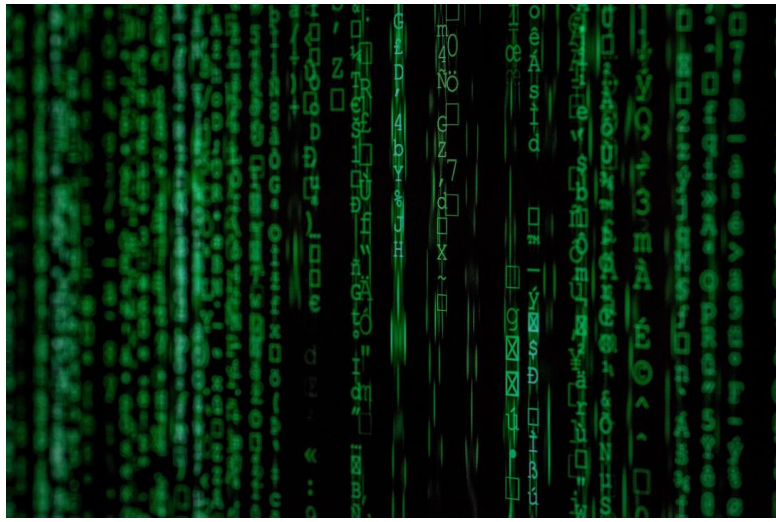


Photo by Markus Spiske on Unsplash

terraform destroy

This is the last command you will use. As the name suggests, it is used to remove infrastructure.

1. Within the scope of the current terraform project, all resources will be terminated
2. Like **`apply`**, the command invokes a **`plan`** detailing how the infrastructure will change and the user must confirm before any resource is destroyed
3. Don’t forget to run this when finished labs or code reviews, otherwise you can run up significant cloud bills!!

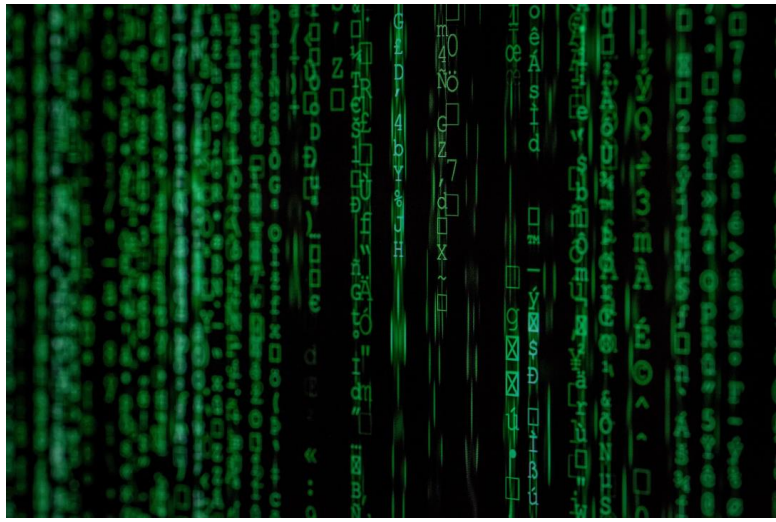


Photo by Markus Spiske on Unsplash

Exercise 1: Simple VM (1 of 8)

Code Overview

Lines 1-10 are configuration to support running Terraform

Lines 12-14 indicate which AWS region will be used

Lines 16-22 provide the details of what is required and how it is named

```
1  terraform {
2    required_version = ">= 0.15.0, < 2.0.0"
3
4    required_providers {
5      aws = {
6        source  = "hashicorp/aws"
7        version = "~> 4.0"
8      }
9    }
10 }
11
12 provider "aws" {
13   region = "us-east-1"
14 }
15
16 resource "aws_instance" "firstEC2" {
17   ami           = "ami-0d6927ccef429da8c"
18   instance_type = "t2.micro"
19   tags = {
20     Name = "first_tf_EC2"
21   }
22 }
23
```

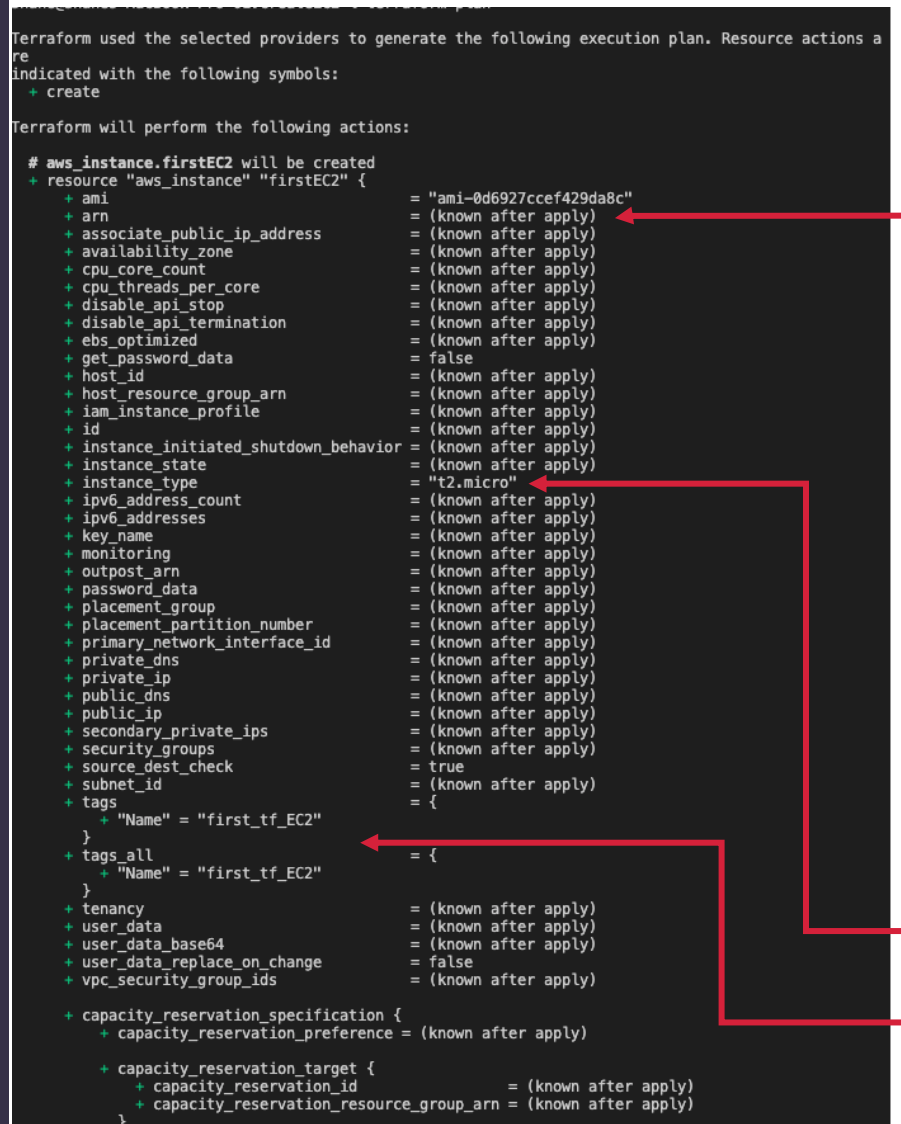
Exercise 1: Simple VM (2 of 8)

`terraform init`

Nothing specified for the backend. AWS provider details provide and **`init`** is completed successfully

```
shane@Shanes-MacBook-Pro 01.CreateEC2 % terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v4.62.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
shane@Shanes-MacBook-Pro 01.CreateEC2 %
```

Exercise 1: Simple VM (3 of 8)



```
Terraform used the selected providers to generate the following execution plan. Resource actions a
re
indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.firstEC2 will be created
  + resource "aws_instance" "firstEC2" {
      + ami                                  = "ami-0d6927ccef429da8c"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Name" = "first_tf_EC2"
        }
      + tags_all                             = {
          + "Name" = "first_tf_EC2"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification {
          + capacity_reservation_preference = (known after apply)

          + capacity_reservation_target {
              + capacity_reservation_id                = (known after apply)
              + capacity_reservation_resource_group_arn = (known after apply)
            }
        }
```

```
resource "aws_instance" "firstEC2" {
  ami           = "ami-0d6927ccef429da8c"
  instance_type = "t2.micro"
  tags = {
    Name = "first_tf_EC2"
  }
}
```

terraform plan

- A lot is unknown when you run plan, just the items that you have configured are known at this point
- This gives you a view of what will change when you apply this terraform
- It will highlight syntax problems and errors
- Later we will use this command to detect changes in state or drift

Exercise 1: Simple VM (4 of 8)

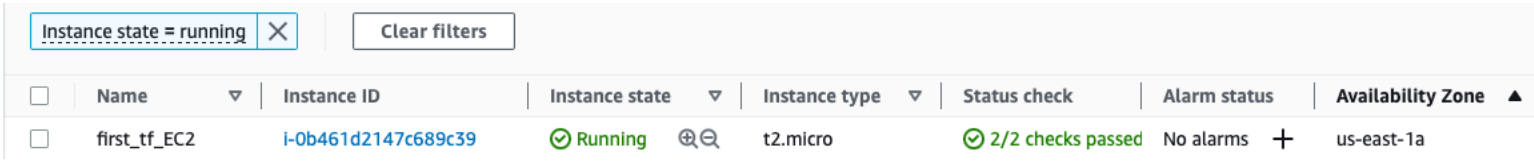
`terraform apply`

Repeats the **`terraform plan`** and asks if you wish to perform these actions

Provides updates on progress with the resource creation, with the instance id of EC2 included in output

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.firstEC2: Creating...
aws_instance.firstEC2: Still creating... [10s elapsed]
aws_instance.firstEC2: Still creating... [20s elapsed]
aws_instance.firstEC2: Still creating... [30s elapsed]
aws_instance.firstEC2: Creation complete after 33s [id=i-0b461d2147c689c39]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
shane@Shanes-MacBook-Pro 01.CreateEC2 %
```

Exercise 1: Simple VM (5 of 8)

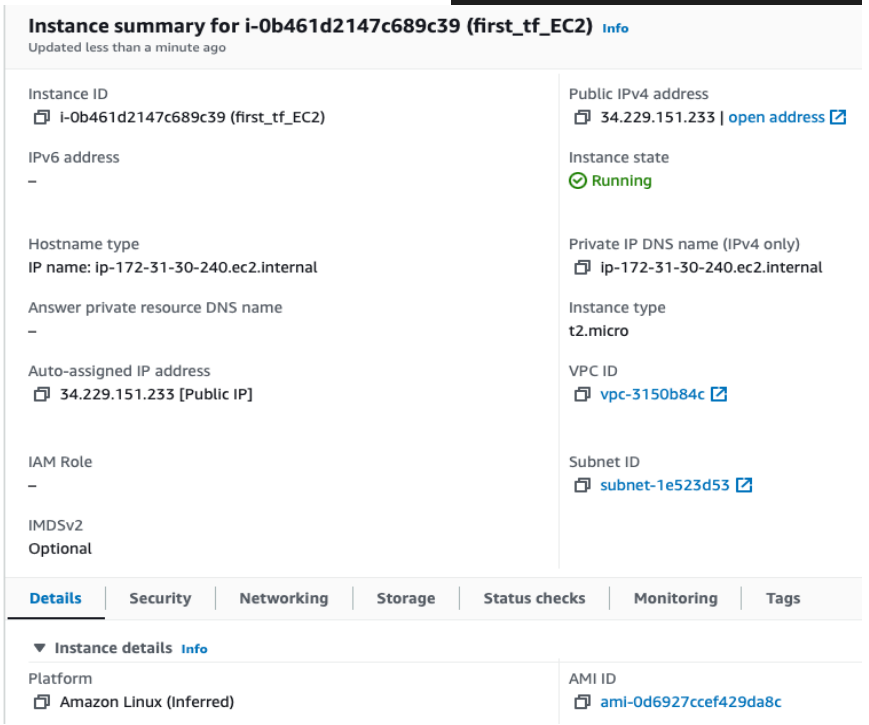



```
resource "aws_instance" "firstEC2" {
  ami           = "ami-0d6927ccef429da8c"
  instance_type = "t2.micro"
  tags = {
    Name = "first_tf_EC2"
  }
}
```

`terraform apply`

Log into EC2 Dashboard to see the EC2 you created.

1. AMI, Type & Name align.
2. Default VPC and Subnet are used.
3. Not possible to access the EC2, as no ports were opened.
4. It exists but does not do anything.



```
shane@Shanes-MacBook-Pro 01.CreateEC2 % terraform destroy
aws_instance.firstEC2: Refreshing state... [id=i-0b461d2147c689c39]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_instance.firstEC2 will be destroyed
  - resource "aws_instance" "firstEC2" {
      - ami                                  = "ami-0d6927ccef429da8c" -> null
      - arn                                  = "arn:aws:ec2:us-east-1:799204530653:instance/i-0b46
1d2147c689c39" -> null
      - associate_public_ip_address          = true -> null
      - availability_zone                    = "us-east-1a" -> null
      - cpu_core_count                       = 1 -> null
      - cpu_threads_per_core                 = 1 -> null
      - disable_api_stop                     = false -> null
      - disable_api_termination              = false -> null
      - ebs_optimized                        = false -> null
      - get_password_data                    = false -> null
      - hibernation                          = false -> null
      - id                                   = "i-0b461d2147c689c39" -> null
      - instance_initiated_shutdown_behavior = "stop" -> null
      - instance_state                       = "running" -> null
      - instance_type                        = "t2.micro" -> null
      - ipv6_address_count                   = 0 -> null
      - ipv6_addresses                       = [] -> null
      - monitoring                           = false -> null
      - placement_partition_number           = 0 -> null
      - primary_network_interface_id         = "eni-0bd140d4799e34d15" -> null
      - private_dns                          = "ip-172-31-30-240.ec2.internal" -> null
      - private_ip                           = "172.31.30.240" -> null
      - public_dns                           = "ec2-34-229-151-233.compute-1.amazonaws.com" -> nul
l
      - public_ip                            = "34.229.151.233" -> null
      - secondary_private_ips                = [] -> null
      - security_groups                      = [
          - "default",
        ] -> null
      - source_dest_check                    = true -> null
      - subnet_id                            = "subnet-1e523d53" -> null
      - tags                                 = {
          - "Name" = "first_tf_EC2"
        } -> null
      - tags_all                             = {
          - "Name" = "first_tf_EC2"
        } -> null
      - tenancy                              = "default" -> null
      - user_data_replace_on_change          = false -> null
      - vpc_security_group_ids               = [
          - "sg-3087ed09",
        ] -> null

      - capacity_reservation_specification {
          - capacity_reservation_preference = "open" -> null
        }

      - credit_specification {
          - cpu_credits = "standard" -> null
        }

      - enclave_options {
          - enabled = false -> null
        }

      - maintenance_options {
          - auto_recovery = "default" -> null
        }

      - metadata_options {
          - http_endpoint               = "enabled" -> null
          - http_put_response_hop_limit = 1 -> null
          - http_tokens                 = "optional" -> null
          - instance_metadata_tags      = "disabled" -> null
        }

      - private_dns_name_options {
          - enable_resource_name_dns_a_record    = false -> null
          - enable_resource_name_dns_aaaa_record = false -> null
          - hostname_type                        = "ip-name" -> null
        }

      - root_block_device {
          - delete_on_termination = true -> null
          - device_name           = "/dev/xvda" -> null
          - encrypted             = false -> null
          - iops                  = 100 -> null
          - tags                  = {} -> null
          - throughput            = 0 -> null
          - volume_id             = "vol-012f727c48d51d5a5" -> null
          - volume_size           = 8 -> null
          - volume_type           = "gp2" -> null
        }
    }

Plan: 0 to add, 0 to change, 1 to destroy.
```

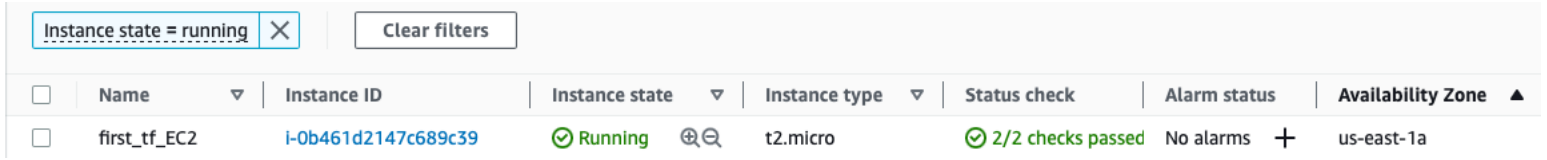
Exercise 1: Simple VM (6 of 8)

`terraform destroy`

Like the reverse of apply, it lets you know everything that will be removed and prompts you to confirm

The instance ID is provided as a reference, with progress and summary included in output

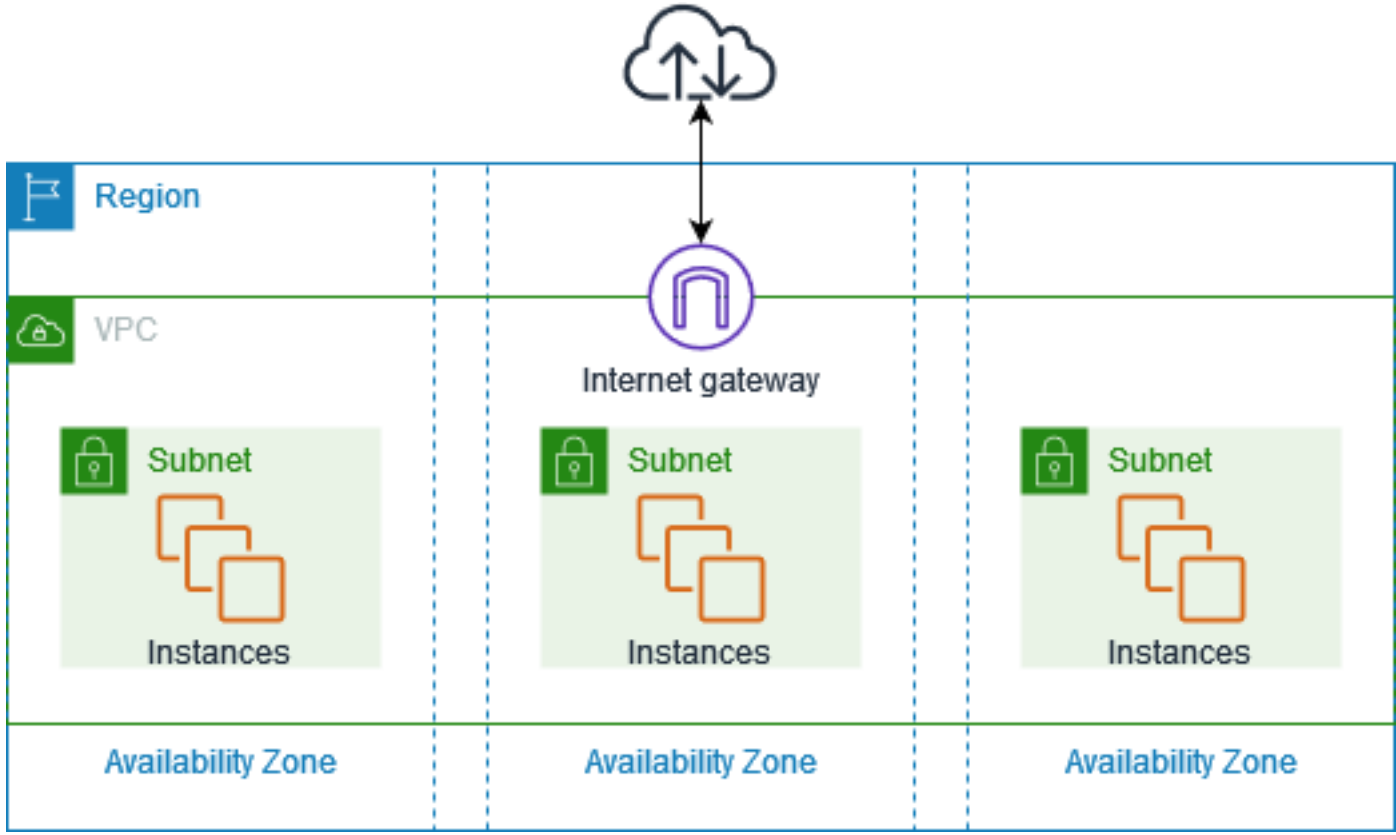
```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.firstEC2: Destroying... [id=i-0b461d2147c689c39]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 10s elapsed]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 20s elapsed]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 30s elapsed]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 40s elapsed]
aws_instance.firstEC2: Destruction complete after 41s

Destroy complete! Resources: 1 destroyed.
```



Exercise 1: Simple VM (7 of 8)

05.AlbWithSubnetWithMultiAzs

1. specify region, which has azs
2. create vpc in this region
3. create subnets, specify vpc and az – min 2, in different azs
4. create security group for alb, specify vpc
5. create alb, specify subnets and security group



05.AlbWithSubnetWithMultiAzs

1. specify region, which has azs
2. create vpc in this region
3. create subnets, specify vpc and az – min 2, in different azs
4. create security group for alb, specify vpc
5. create alb, specify subnets and security group

Add the following next

- aws_lb_target_group – specify vpc_id
- aws_lb_ listener– specify alb.arn
- aws_lb_listener_rule – specify aws_lb_ listener

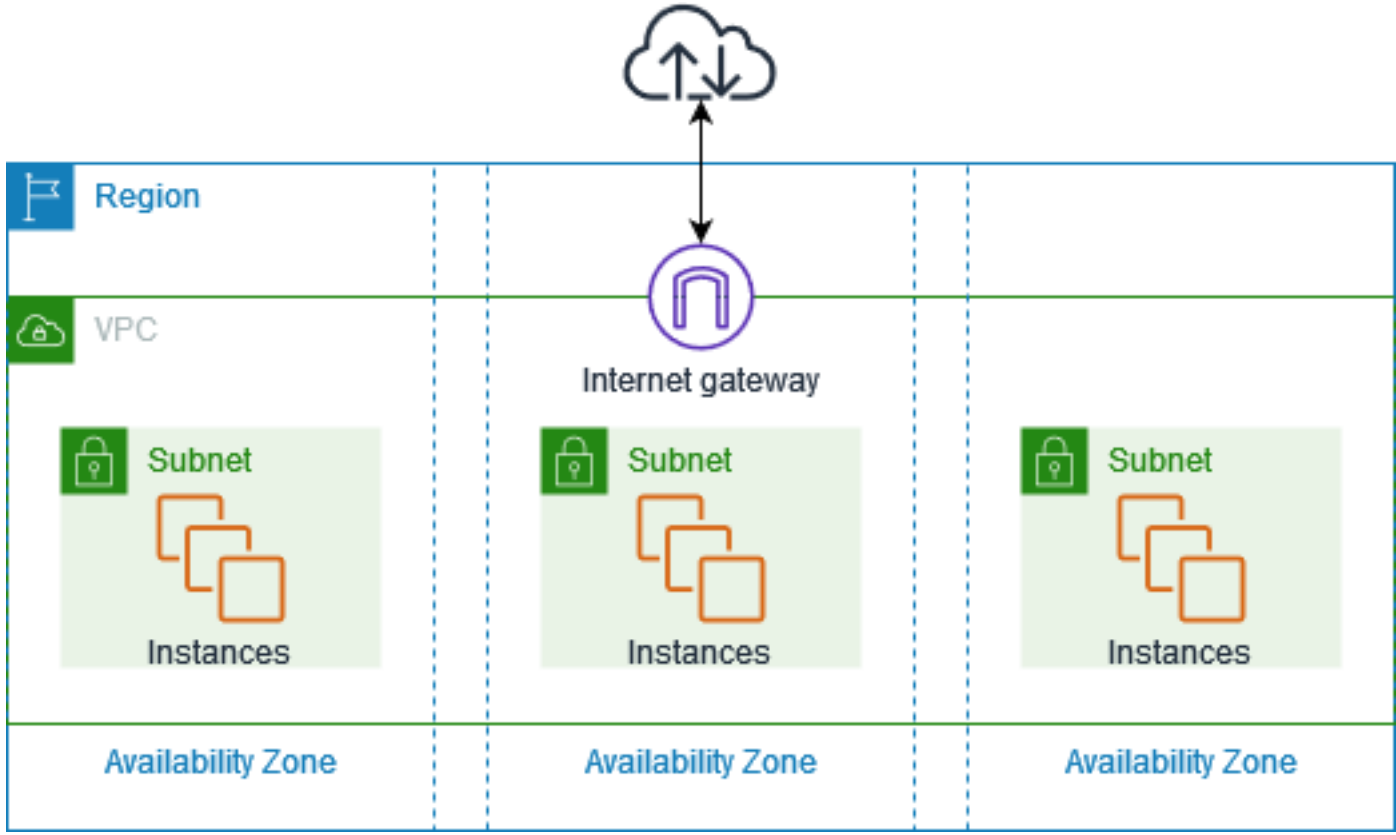
- aws_ssm_parameter
- template_file
- aws_route_table – specify vpc
- aws_route_table_association – change from subnet to gateway here removed this as there was a conflict and may not be needed.

- aws_security_group (not alb)
- aws_launch_configuration
- aws_autoscaling_group

Later

- aws_availability_zones – look at how to improve subnet creation based on azs

Exercise 1: Simple VM (8 of 8)



Wrap up

- Q & A
- What's 1 thing you learned?
- What's 1 challenge?

References

- Images [Slide 1] used under license from Microsoft PowerPoint stock images.
- Image [Slide 16] 2020 "Woman Sitting on Bench Looking at Mountains" by Yan Krukau used under free license from Pexels.